

Functional Programming Exam:

Thursday, August 14, 2025

- The exam duration is five hours.
- There are four questions. To obtain full marks you must answer all the subquestions satisfactorily.
- You are allowed to use books, lecture notes, lecture slides, hand-ins, solutions to assignments, calculators, computers, software etc. during the examination. This includes F# compilers and IDEs.
- You may **NOT** use the internet in any way other than WiseFlow for the duration of the exam and you must be connected to WiseFlow, and its monitoring software, at all times.
- You **MUST** use the ITU++ WiFi network. Make sure to forget any networks that you may accidentally join by mistake and turn off any mobile devices so that you don't accidentally join a hot spot.
- You may **NOT** use any form of code generators like Visual Studio CoPilot, ChatGPT or equivalent. All code you hand in must either be in the template we provide, or written by you yourself. The use of any such tool is grounds for disciplinary action. Standard auto-completion like Intellisense is ok.
- You may **NOT** communicate with anyone else during the exam, which includes using chat clients, phones, hand gestures, smoke signals, etc. The exception is the invigilators if you have a question.
- All forms of software that can be used to communicate, like Steam, Messenger, Discord, Teams, etc, and any code generation tools (CoPilot, LLMs like ChatGPT, etc) must be completely switched off for the entire exam. It's not enough not to use them, they must be switched off completely. To be safe, restart your computer before the exam and make sure that no banned programs are running.
- You are (unless otherwise instructed) allowed to use the .NET library including the modules described in the book, e.g., `List`, `Set`, `Map` etc.
- If a question requires you to define a function, then you may (unless otherwise instructed)
 - use that function in subsequent subquestions, even if you have not managed to define it. Providing the signature of the missing function will help in such cases.
 - define as many helper functions as you want, but you must define the required function so that it has exactly the intended type and effect, and the helper functions must follow the constraints, that the question requires. A tail recursive assignment may, for instance, be coded as a non recursive main function that calls a tail recursive auxiliary function.
- Unless explicitly stated you must provide functional solutions without side effects
- You are required to use the provided code project `FPEXam2025` as a basis for your submission and you should **only hand in** the `Exam.fs` file. No other file, and **absolutely not a zip file of the entire project**. The project includes everything you need to run as an independent project. Any helper functions that we provide in the `Exam.fs` file may also be part of your submission.
- If an assignment asks that you write a function `isEven : int -> bool`, for instance, then there is a corresponding `let isEven _ = failwith "not implemented"` in `Exam.fs`. You may change these functions (changing a `let` to a `let rec` for instance) as long as their signatures match those given in the assignment. In this case that could be `let isEven x = x % 2 = 0`.

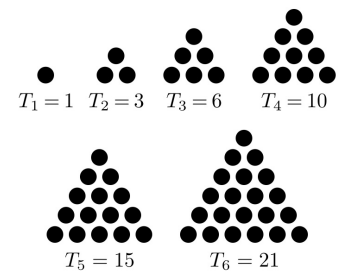
You MUST include explanations and comments to support your solutions for the questions that require them. You simply write them as comments around your code.

Your entire exam hand-in MUST be made by yourself and yourself alone It is illegal to make the exam answers as group work or to enlist the help of others in any way. This includes using solutions or code found online, or tools that write code for you (such as CoPilot or ChatGPT).

Your solution MUST compile. We reserve the right to fail any submission that does not meet this minimal requirement.

1: Triangle Numbers (25%)

The triangle numbers are a sum of consecutive natural numbers. such the n th triangle number T_n is calculated as $1 + 2 + 3 + \dots + n$. They can graphically be represented as a growing triangle of dots where the number of dots represents the triangle number.



Source: Wikipedia

Question 1.1

Create a recursive, but not tail recursive, function `triangleNumber` of type `int -> int list` that given a number `n` returns a list containing the `n` first triangle numbers in increasing order.

For full credit your function must have at most linear complexity.

Hint: Depending on your solution, you may need to use `List.rev`

Examples:

```
> triangleNumber 1
- val it: int list = [1]

> triangleNumber 2
- val it: int list = [1; 3]

> triangleNumber 6
- val it: int list = [1; 3; 6; 10; 15; 21]
```

Question 1.2

Create a tail-recursive function `triangleNumberAcc` using an accumulator (not a continuation) that behaves exactly the same as `triangleNumber` for all possible valid inputs.

For full credit your function must have at most linear complexity.

Question 1.3

The function `List.sum` sums all elements in a list, so for example `List.sum [1;2;3;4;5;6] = 21`.

Create a non-recursive function `triangleNumberList` using only `List.sum` and list comprehension that behaves exactly the same as `triangleNumber` and `triangleNumberAcc` for all possible inputs.

For full credit your function must have at most quadratic complexity.

Hint: You will most likely need nested list comprehension here.

Question 1.4

Create a function `sequence` of type `(int -> 'a -> 'a) -> 'a -> int -> 'a list` that given a function `f`, an initial value `i` and a size `n` returns list `[S1; S2; S3; ...; Sn]` where `S1 = i`, `S2 = f 2 S1`, `S3 = f 3 S2`, up to `Sn = f n Sn-1`

Your triangle number functions from Questions 1.1 and 1.2 can, for instance, be written as `sequence (+) 1`.

For full credit your function must have at most linear complexity.

Examples:

```
> sequence (+) 1 1
val it: int list = [1]

> sequence (+) 1 2
val it: int list = [1; 3]

> sequence (+) 1 6
val it: int list = [1; 3; 6; 10; 15; 21]

> sequence (fun x acc -> acc + string (char (x + 64))) "A" 10
val it: string list =
["A"; "AB"; "ABC"; "ABCD"; "ABCDE"; "ABCDEF"; "ABCDEFG"; "ABCDEFGH";
"ABCDEFGHI"; "ABCDEFGHIJ"]
```

Question 1.5

Using the `sequence` function from Q1.4, create a non recursive function `alternatingTriangle` of the type `int -> int list` that works exactly like triangle numbers except that even numbers are subtracted rather than added.

$$\begin{aligned} \text{Triangle number } T_n &= 1 + 2 + \dots + n \\ \text{Alternating triangle number } A_n &= \begin{cases} 1 - 2 + 3 - 4 + \dots + n & \text{if } n \text{ is odd} \\ 1 - 2 + 3 - 4 + \dots - n & \text{if } n \text{ is even} \end{cases} \end{aligned}$$

Examples:

```
> alternatingTriangle 1
- val it: int list = [1]

> alternatingTriangle 2
- val it: int list = [1; -1]

> alternatingTriangle 10
- val it: int list = [1; -1; 2; -2; 3; -3; 4; -4; 5; -5]

> alternatingTriangle 15
- val it: int list = [1; -1; 2; -2; 3; -3; 4; -4; 5; -5; 6; -6; 7; -7; 8]
```

2: Code comprehension (25%)

Consider the following functions

```
let rec foo a =  
  match a with  
  | []      -> []  
  | [x]     -> []  
  | x :: y :: xs -> (x, y) :: foo (y :: xs)  
  
let rec bar f a =  
  match a with  
  | (x, y) :: xs when f x y -> bar f xs (* 1 *)  
  |      :: xs      -> false  (* 2 *)  
  | []              -> true   (* 3 *)  
  
let baz = foo >> bar (fun x y -> (x + y) % 2 <> 0)
```

Question 2.1

- Give two different input lists, of length strictly greater than five, that when you call `baz` give different results.
- What do the functions `foo`, `bar` and `baz` do? Focus on what they do rather than how they do it.
- What would be appropriate names for functions `foo`, `bar` and `baz`?
- In the `foo` function, what would happen if you replaced the match `[x]` with `[_]`? Motivate your answer.

Question 2.2

- What would happen if you swap lines `(* 1 *)` and `(* 2 *)` in the `baz` function?
- What would happen if you swap lines `(* 2 *)` and `(* 3 *)` in the `baz` function?

Question 2.3

Create a single function `baz2`, possibly with at most one internal function, that behaves exactly the same way as `baz` for all possible inputs. Your function may be recursive, tail recursive, or not recursive, and you may use functions from the standard library. For full credit your code must be clean and simple.

Question 2.4

The function `foo` from Question 2.1 is not tail recursive. Explain why. To make a compelling argument you must evaluate a function call of the function, similarly to what is done in Chapter 1.4 of HR, and reason about that evaluation. You need to make clear what aspects of the evaluation tell you that the function is not tail recursive. Keep in mind that all steps in an evaluation chain must evaluate to the same value ($(5 + 4) * 3 \rightarrow 9 * 3 \rightarrow 27$, for instance).

Question 2.5

Create a function `fooTail` that behaves the same way as `foo` for all possible inputs but which is tail recursive and coded using continuations.

3: Stores and locks (25%)

In this exercise we will create a store, containing a single value, and a lock where the intuition is that anyone can always read the current value from the store but you can also lock the store to ensure that only you can change its value, after which you can release the lock to allow others to change it. The value of an unlocked store can always be modified. We will call the participants who vie for control of the store *processes*.

Question 3.1

We will use the following type for a store

```
type 'a store = {  
  data : 'a  
  owner : int option  
}
```

- `data` contains the current value in the store
- the store is unlocked if `owner` is `None`
- the store is locked by the process with id `pid` if `owner` is `Some pid`

Create the following functions

- `newStore` of type `'a -> 'a store` that given a value `v` creates a store with `data` set to `v` and `owner` set to `None`
- `lock` of type `int -> 'a store -> 'a store` that given a process id `pid` and a store `st` and
 - if `owner` in `st` is `None` returns `st` with `owner` set to `Some pid`
 - if `owner` in `st` is `Some pid2` prints the string "process <pid> tried to lock but the lock is held by <pid2>", and returns `st`
- `unlock` of type `int -> 'a store -> 'a store` that given a process id `pid` and a store `st` and
 - if `owner` in `st` is `Some pid` returns `st` with `owner` set to `None`
 - if `owner` in `st` is `Some pid2`, where `pid` is not equal to `pid2`, prints the string "process <pid> tried to unlock but the lock is held by <pid2>", and returns `st`
 - if `owner` in `st` is `None` prints the string "process <pid> tried to unlock but no one holds the lock", and returns `st`

Examples:

```
> 0 |> newStore |> lock 0  
- val it: int store = { data = 0; owner = Some 0 }  
  
> 0 |> newStore |> lock 0 |> unlock 0  
- val it: int store = { data = 0; owner = None }  
  
> 0 |> newStore |> lock 0 |> lock 1  
process 1 tried to lock but the lock is held by 0  
- val it: int store = { data = 0; owner = Some 0 }  
  
> 0 |> newStore |> lock 0 |> unlock 1  
process 1 tried to unlock but the lock is held by 0  
- val it: int store = { data = 0; owner = Some 0 }  
  
> 0 |> newStore |> unlock 0  
process 0 tried to unlock but no one holds the lock  
- val it: int store = { data = 0; owner = None }
```

Question 3.2

Create the following functions

- `read` of type `'a store -> 'a` that given a store `st` returns its `data` regardless of whether or not it is locked.
- `write` of type `int -> 'a -> 'a store -> 'a store` that given a process id `pid`, a new data `value` and a store `st` and
 - `owner` is equal to `Some pid2`, where `pid` is not equal to `pid2`, prints the string `"process <pid> tried to write <value> but the lock is held by <pid2>"`, and returns `st`
 - returns `st` with `data` set to `value`, otherwise.
- `isLocked` of type `'a store -> bool` that given a store `st` returns
 - `true` if `owner` is `Some pid`
 - `false` if `owner` is `None`

Examples:

```
> 0 |> newStore |> read
- val it: int = 0

> 0 |> newStore |> write 0 1 |> read
- val it: int = 1

> 0 |> newStore |> write 0 1
- val it: int store = { data = 1; owner = None }

> 0 |> newStore |> lock 0 |> write 0 1
- val it: int store = { data = 1; owner = Some 0 }

> 0 |> newStore |> lock 1 |> write 0 1
process 0 tried to write 1 but the lock is held by 1
- val it: int store = { data = 0; owner = Some 1 }

> 0 |> newStore |> isLocked
- val it: bool = false

> 0 |> newStore |> lock 1 |> isLocked
- val it: bool = true
```

Question 3.3

Important: This is the hardest question in Q3. You can still get full credit for Q3.4 and Q3.5 even if you do not solve this one at all. You will need the `message` type, the `storeServer` type, and the `inbox` function defined below. Regardless, we highly recommend you read both 3.3 and 3.4 as they do tie together.

For this question you may not break the abstraction of the `store` type but must use the functions you made in Q3.1 and Q3.2 to access and modify the stores.

We will make a mailbox that tracks a store and that allows clients to lock, unlock, read from, and write to the store.

- if a process tries to lock an already locked store then that lock request is placed in a pending queue
- when the store is unlocked
 - the next process from the pending queue reestablishes the lock
 - if there are no processes in the pending queue the lock is released.
- it is still possible to fail by unlocking an unlocked store or writing to a store locked by another process.
- all reads succeed

We use the following code skeleton

```
type 'a message =
| Lock   of int * AsyncReplyChannel<unit>
| Unlock of int
| Read   of AsyncReplyChannel<'a>
| Write  of int * 'a

type 'a storeServer = Store of MailboxProcessor<'a message>

let inbox x (mbox : MailboxProcessor<'a message>) =
    let rec messageLoop (st : 'a store) (pending : (int * AsyncReplyChannel<unit>) list) =
        failwith "not implemented"
    messageLoop (newStore x) []
```

The type 'a message allows processes to read from and write to stores as well as locking and unlocking them.

- Lock(pid, rc) informs the mailbox that process pid would like to lock the store and that it should return a unit message () on the return channel rc when the lock is established.
- Unlock pid informs the mailbox that process pid would like to unlock the store
- Read(rc) informs the mailbox to return the current data of the store on the return channel rc.
- Write(pid, value) informs the mailbox that process pid would like to update the data of the store to value.

The main loop messageLoop takes the current store st and a list of pairs pending as an argument where each pair contains the id of a process, and its return channel. The list represents the processes who want to lock the store, but where the store is already locked.

The messageLoop function itself should use the functions you created in Q3.1 and Q3.2 and :

- Receive the next message msg from the inbox
- if msg is Lock(pid, rc) and
 - st is unlocked then have pid lock st and send () back on rc
 - st is locked, then add pid and rc to the pending queue (front or back does not matter)
- if msg is Unlock pid then unlock st and
 - if pending has the form (pid, rc)::pending', have pid lock st, send () back on rc, and continue with pending'.
 - if pending is [] then continue without locking the store
- if msg is Read rc then send the current data of st back on rc
- if msg is Write(pid, value) then continue with st, but where the process pid has changed data to value.

Examples: You will find examples in Q3.4

Question 3.4

Create the following functions

- `createStore` of type `'a -> 'a storeServer` that given an initial value `v` initialises a new mailbox as a `'a storeServer` with `v` as its initial value.
- `storeLock` of type `int -> 'a storeServer -> unit` that given a process id `pid` and a store server `st` sends a message to `st` that the process `pid` would like to lock `st` and returns once `st` has sent a message back along a return channel.
- `storeUnlock` of type `int -> 'a storeServer -> unit` that given a process id `pid` and a store server `st` sends a message to `st` that the process `pid` would like to unlock `st`. The function returns immediately without waiting for a reply on a return channel.
- `storeRead` of type `'a storeServer -> 'a` that given a store server `st` sends a message to `st` that the process `pid` would like read the current data value and returns the value sent along the return channel.
- `storeWrite` of type `int -> 'a -> 'a storeServer -> unit` that given a process id `pid`, a value `value`, and a store server `st` sends a message to `st` that the process `pid` would like to write the value `value` to `st`. The function returns immediately without waiting for a reply on a return channel.

Examples:

```
let st1 = createStore 0
let st2 = createStore 0
let st3 = createStore 0

> storeLock 0 st1; storeWrite 0 42 st1; storeUnlock 0 st1; storeRead st1
- val it: int = 42

> storeLock 0 st2; storeWrite 0 42 st2; storeUnlock 0 st2; storeWrite 1 50 st2; storeRead st2
- val it: int = 50

> storeLock 0 st3; storeWrite 1 42 st3; storeRead st3
process 1 tried to write 42 but the lock is held by 0
- val it: int = 0
```

Question 3.5

Create the following functions

- `inc` of type `int -> int storeServer -> unit` that given a process id `pid` and a store server `st` has the process `pid` lock `st`, then read the current data of `st`, then write `data + 1` to `st`, and finally unlock `st`
- `countTo` of type `int -> int` that given a number of processes `size`
 - Creates a new storeServer `st`
 - spawns `size` processes with unique process ids (we suggest `1..size`)
 - Runs `inc` for each process in parallel on the store server `st`
 - when all processes are finished, read and return the current data from `st`.

Examples:

```
> countTo 10 (* Don't go higher than ten for performance reasons due to default thread pool sizes *)
- val it: int = 10
```


4: Tic-tac-toe (25%)

Tic-tac-toe is a simple game where two players take turns placing crosses and circles respectively on a 3x3 grid. The first player to get three in a row on either one of the three horizontal lines, one of the three vertical lines, or one of the two diagonal lines wins the game. In the image on the right, the player playing with circles has won the game.



Source: Wikipedia

Important: There are many correct answers to what is a good representation of a game, but it should be easy to use with the functions that you create below. It is a good idea to read and understand the Questions 4.1 and 4.2 before you commit to a design.

Question 4.1

- Create the types `row` and `col` that represents names of rows and columns on the board respectively.
 - Create the values `topRow`, `midRow` and `botRow` to represent the top, the middle, and the bottom row of the board respectively.
 - Create the values `leftCol`, `midCol` and `rightCol` to represent the left, the middle, and the right column of the board respectively.
- Create a type `player` that represents if a player is playing crosses or circles.
 - Create a value `X` representing the player playing crosses
 - Create a value `O` representing the player playing circles
- Create a type `board` that represents a current state of the game. It must be able to tell what tiles the players have placed on the board. You can use the `player` type to indicate if a cross or a circle has been placed on a square.

The following notation describes a board where `X` and `O` represents crosses and circles respectively

```
-----  
|  O  | X  | X  |  
-----  
|  O  | O  | X  |  
-----  
|  X  | O  | O  |  
-----
```

- Create a value `newGame` of type `board` that represents an empty board with no tiles placed on the board.

Examples:

```
> newGame
```

```
- val it : board =
```

```
-----  
|  |  |  |  
-----  
|  |  |  |  
-----  
|  |  |  |  
-----
```

Question 4.2

For this question we will be using the `Result` type to report the following error.

```
type error =  
| PlayerTurn of player  
| SquareTaken of row * col * player
```

where

- `PlayerTurn p` means that it is player `p` to move, but the other player tried to place a tile.
- `SquareTaken(r, c, p)` means that we tried to place a tile on row `r` and column `c` but that square already has a tile placed on it by player `p`.

We will use the following type `state` to represent the different states a game can be in.

```
type state =  
| Running of player * board  
| Win of player * board  
| Draw of board
```

where

- `Running(p, b)` means that the game is still being played, that it is player `p` to go next, and the current board configuration is `b`
- `Win(p, b)` means that the game is over and player `p` won and the finishing board was `b`
- `Draw b` means that the game is over and the game ended in a draw and the finishing board was `b`.

Create a function `doNextMove` of type `player -> row -> col -> state -> Result<state, error>` that given a player `p`, a row `r`, a column `c`, and a state `st` attempts to have `p` place their tile on row `r` and column `c` and returns

- `Ok (Win (p, b))` if `st` is equal to `Win(p, b)`
- `Ok (Draw b)` if `st` is equal to `Draw b`
- If `st` is equal to `Running(p', b)` returns
 - `Error(PlayerTurn p')` if it is player `p'` to move and `p` is not equal to `p'`
 - `Error(SquareTaken(r, c, p'))` if the player `p'` has a tile on row `r` and column `c` on the board `b`
 - `Ok (Win (p, b'))` if player `p` placing a tile on row `r` and column `c` wins on the final board `b'`
 - `Ok (Draw b')` if player `p` placing a tile on row `r` and column `c` results in a draw on the final board `b'`
 - `Ok (Running(p', b'))` otherwise, where `p'` is the other player than `p` and where `b'` is the same as `b` but where player `p` has placed a tile on row `r` and column `c`.

Examples:

```
> Running(X, empty) |>
  doMove X midRow midCol
```

```
- Ok(Running(0,
  -----
  | | | |
  |---|---|---|
  |   X   |
  |---|---|---|
  | | | |
  -----
))
```

```
> Running(X, empty) |>
  doMove X midRow midCol |>
  Result.bind (doMove 0 topRow leftCol)
```

```
- Ok(Running(X,
  -----
  | 0 | | |
  |---|---|---|
  |   X   |
  |---|---|---|
  | | | |
  -----
))
```

```
> Running(X, empty) |>
  doMove X midRow midCol |>
  Result.bind (doMove X topRow leftCol)
- Error(PlayerTurn 0)
```

```
> Running(X, empty) |>
  doMove X midRow midCol |>
  Result.bind (doMove 0 midRow midCol)
- Error(SquareTaken(midRow, midColumn, X))
```

```
> Running(X, empty) |>
  doMove X midRow midCol |>
  Result.bind (doMove 0 topRow leftCol) |>
  Result.bind (doMove X topRow rightCol) |>
  Result.bind (doMove 0 midRow leftCol) |>
  Result.bind (doMove X botRow leftCol)
```

```
- Ok(Win(X,
  -----
  | 0 | | X |
  |---|---|---|
  | 0 | X | |
  |---|---|---|
  | X | | |
  -----
))
```

```
> Running(X, empty) |>
  doMove X midRow midCol |>
  Result.bind (doMove 0 topRow leftCol) |>
  Result.bind (doMove X topRow rightCol) |>
  Result.bind (doMove 0 botRow leftCol) |>
  Result.bind (doMove X midRow leftCol) |>
  Result.bind (doMove 0 midRow rightCol) |>
  Result.bind (doMove X topRow midCol) |>
  Result.bind (doMove 0 botRow midCol) |>
  Result.bind (doMove X botRow rightCol)
```

```
- Ok(Draw(
  -----
  | 0 | X | X |
  |---|---|---|
  | X | X | 0 |
  |---|---|---|
  | 0 | 0 | X |
  -----
))
```

Question 4.3

For this assignment we will be using a state monad to hide the tic-tac-toe board and its state. The state monad you will be working on is very similar to the one that you used for the yellow assignments in your interpreter, but the state is much simpler (the `state` type from Question 4.2).

```
type ticTacToeMonad<'a> = TTT of (state -> Result<'a * state, error>)

let ret x = TTT (fun h -> (Ok (x, h)))
let fail err = TTT (fun _ -> Error err)
let bind f (TTT a) =
  TTT (fun h ->
    match a h with
    | Ok (x, h') ->
      let (TTT g) = f x
      g h'
    | Error err -> Error err)

let (>=) a f = bind f a
let (>>=) a b = a >= (fun _ -> b)

let evalTTT (TTT f) l = f (Running(empty, X))
```

Create the following functions where successful results, or errors, should be returned in the state monad as appropriate. Moreover failure, means monadic fail (using `fail`) and not `failwith`.

- `doMove2` of type `player -> row -> col -> ticTacToeMonad<unit>` that given a player `p`, a row `r` and a column `c` has player `p` place a tile on the board and fails if this is not possible.
- `gameOver` of type `ticTacToeMonad<bool>` that returns (monadically) `true` if the game is over (a draw or someone has won), and `false` otherwise.
- `getBoard` of type `ticTacToeMonad<board>` that returns (monadically) the current board of the state.

Important: You cannot use monadic operators, like `bind` or `ret`, for these functions as you must break the abstraction of the tic-tac-toe monad to implement them, but we include them here for debugging purposes, and you will need them for Question 4.4.

Examples:

```
(* This is the same game as the last example of Q4.3 that ended in a draw *)
let drawGame =
  doMove2 X midRow midCol >>=
  doMove2 O topRow leftCol >>=
  doMove2 X topRow rightCol >>=
  doMove2 O botRow leftCol >>=
  doMove2 X midRow leftCol >>=
  doMove2 O midRow rightCol >>=
  doMove2 X topRow midCol >>=
  doMove2 O botRow midCol >>=
  doMove2 X botRow rightCol

> evalTTT gameOver
- val it: Result<bool,error> = Ok false

> evalTTT (drawGame >>= gameOver)
- val it: Result<bool,error> = Ok true

> evalTTT (drawGame >>= getBoard)
- Ok(
  | 0 | X | X |
  | X | X | O |
  | 0 | O | X |
  )
```

Question 4.4

For this assignment you may, if you want to, use computation expressions in which case you will need the following definitions.

```
type TicTacToeBuilder() =  
    member this.Bind(f, x)      = bind x f  
    member this.Return(x)      = ret x  
    member this.ReturnFrom(x) = x  
    member this.Combine(a, b) = a >>= (fun _ -> b)  
  
let ttt = new TicTacToeBuilder()
```

You may also solve the assignment using monadic operators, but you may not break the abstraction of the state monad (you may not pattern match on anything of type `ticTacToeMonad`).

Create a function `playGame` of type `(player * row * col) list -> ticTacToeMonad<board>` that given a list of moves `moves`, where each move contains of a player to move and a row and a column where they place their tile, performs the moves in `moves` until either the list runs out, or the game is over, at which point it returns the ending board.

Examples:

```
> [(X, midRow, midCol)] |> playGame |> evalTTT  
  
-----  
| | | |  
|---|---|---|  
- Ok(| | X | | )  
|---|---|---|  
| | | |  
-----  
  
> [(X, midRow, midCol);  
    (O, topRow, leftCol);  
    (X, topRow, rightCol);  
    (O, midRow, leftCol);  
    (X, botRow, leftCol);  
    (O, midRow, midCol) (* This last move will never be played because the game is over*)] |>  
    playGame |>  
    evalTTT  
  
-----  
| 0 | | X |  
|---|---|---|  
- Ok(| 0 | X | | )  
|---|---|---|  
| X | | |  
|---|---|---|  
| | | |  
-----
```

Question 4.5

Create the parsers `parsePlayer`, `parseRow` of type `Parser<player>`, `Parser<row>` and `parseCol` of type `Parser<col>` where

- `parsePlayer` parses the string "X" to X and "0" to 0 respectively.
- `parseRow` parses the string "topRow", "midRow", and "botRow", to `topRow`, `midRow`, and `botRow` respectively.
- `parseColumn` parses the string "leftCol", "midCol", and "rightCol", to `leftCol`, `midCol`, and `rightCol` respectively.

Create a parser `parseMove` of type `Parser<player * row * col>` that parses a string of the form "Player <player to be parsed by `parsePlayer`> places a tile on row <row to be parsed by `parseRow`> and column <column to be parsed by `parseColumn`>"

Create a parser `parseMoves` of type `Parser<(player * row * col) list>` that parses a, possibly empty, sequence of moves, as parsed by `parseMove`, separated by one or more white spaces.

Examples:

```
> run parseMoves
"Player X places a tile on row midRow and column midCol
Player 0 places a tile on row topRow and column leftCol
Player X places a tile on row topRow and column rightCol
Player 0 places a tile on row midRow and column leftCol
Player X places a tile on row botRow and column leftCol"
|> Result.map (playGame >> evalTTT >> Result.toOption >> Option.get)
```

```
-----
| 0 |  | X |
|---|---|---|
- Ok(| 0 | X |  |)
|---|---|---|
| X |  |  |
|---|---|---|
```